W5

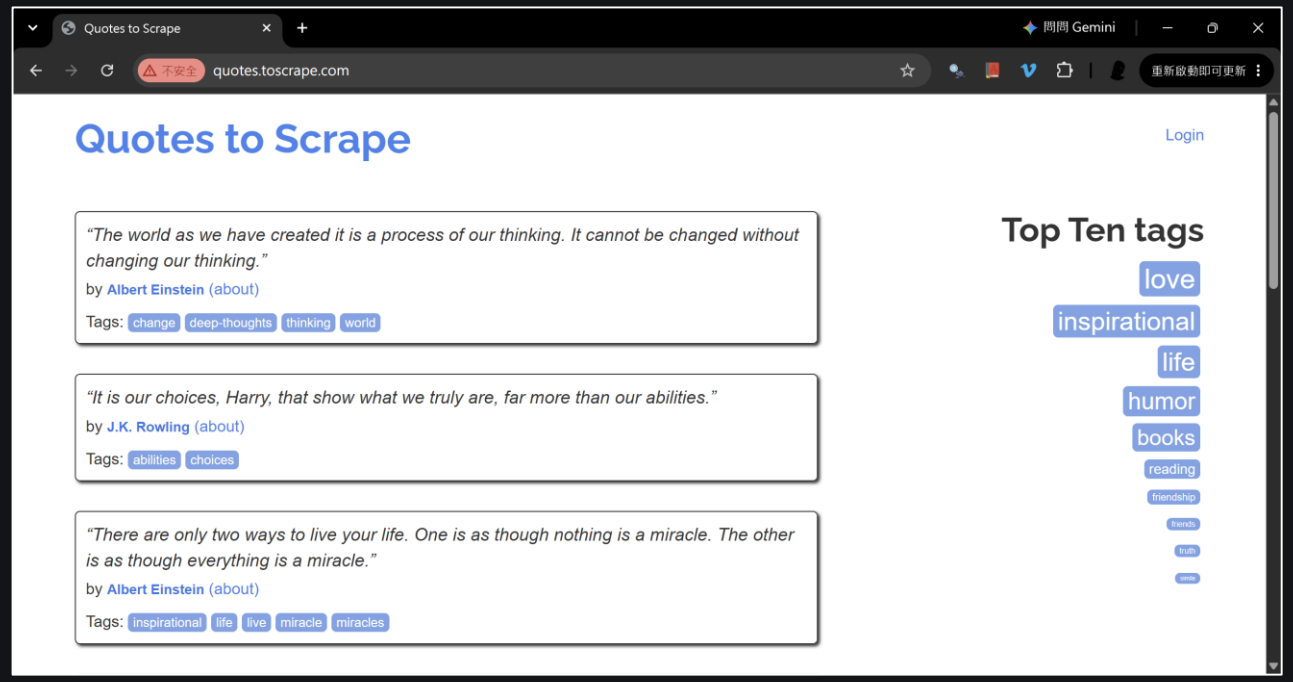
正規表達式與文字資料擷取

樣式語法 · re 模組 · 從網頁文字抽出結構化欄位

吳彥霖 博士

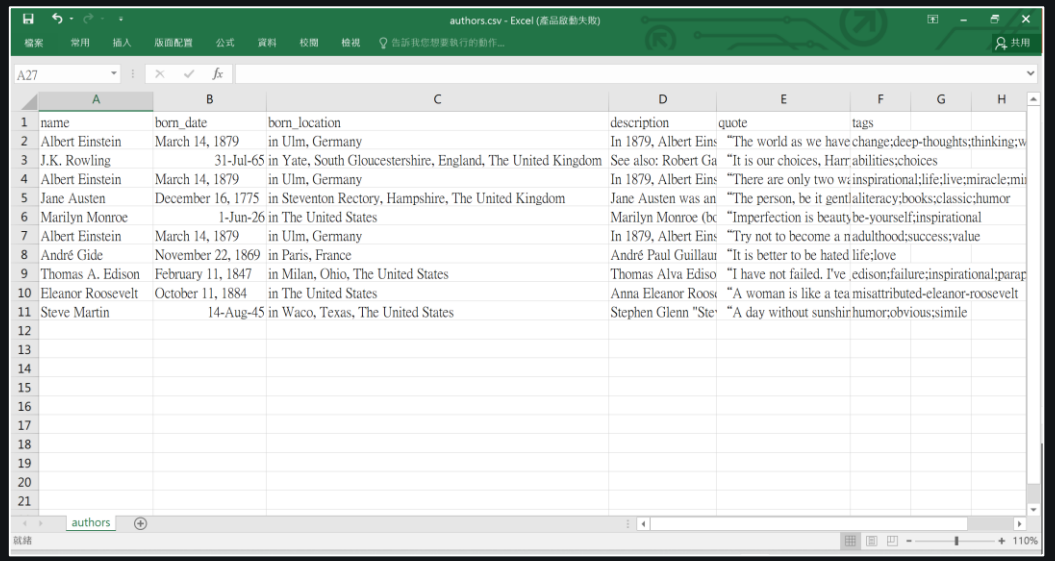
Python 爬蟲實務應用 · 第 5 週 · 10/13

課堂練習 - 1

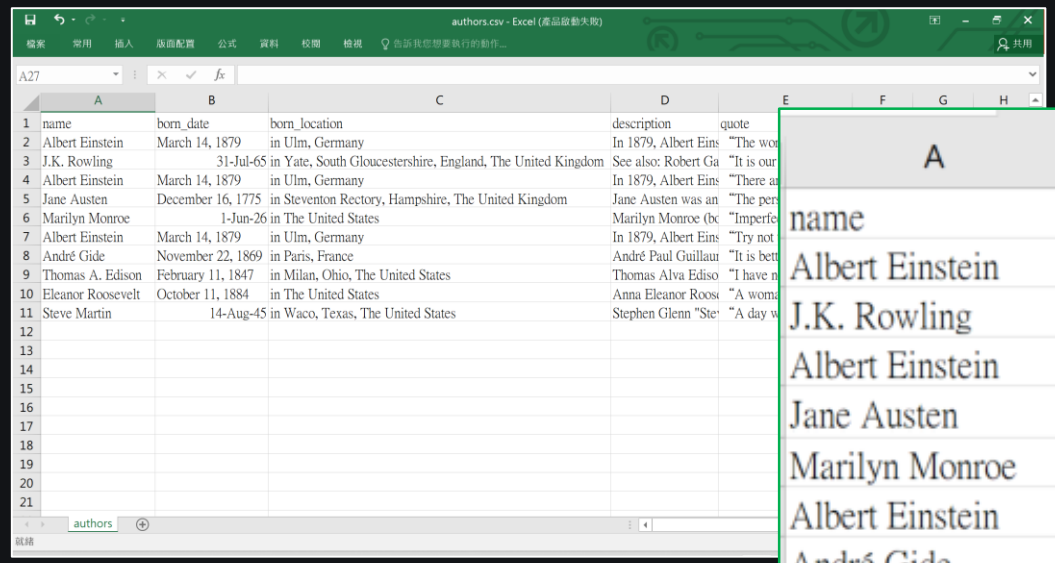


http://quotes.toscrape.com/

- 利用 BeautifulSoup 模組，爬取 Quotes to Scrape 首頁各名言與作者資訊，且將資訊存成 authors.csv 。
  欄位名稱 : name、born_date、born_location、quote、tags 。



爬到文字，不等於拿到可應用的資料

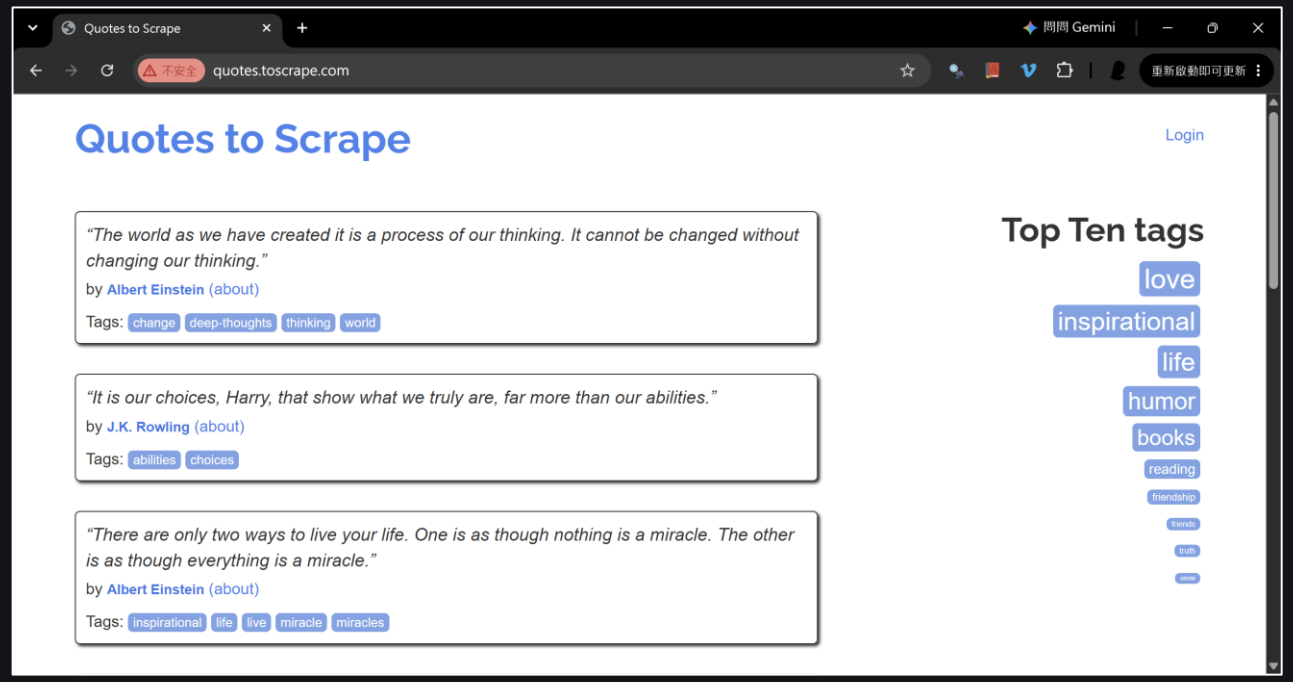


	A	B	C	D	E
1	name	born_date	born_location	description	quote
2	Albert Einstein	March 14, 1879	in Ulm, Germany	In 1879, Albert Eins	"The wor
3	J.K. Rowling	31-Jul-65	in Yate, South Gloucestershire, England, The United Kingdom	See also: Robert Ga	"It is our
4	Albert Einstein	March 14, 1879	in Ulm, Germany	In 1879, Albert Eins	"There ar
5	Jane Austen	December 16, 1775	in Steventon Rectory, Hampshire, The United Kingdom	Jane Austen was an	"The pers
6	Marilyn Monroe	1-Jun-26	in The United States	Marilyn Monroe (bo	"Imperfe
7	Albert Einstein	March 14, 1879	in Ulm, Germany	In 1879, Albert Eins	"Try not
8	André Gide	November 22, 1869	in Paris, France	André Paul Guillaur	"It is bett
9	Thomas A. Edison	February 11, 1847	in Milan, Ohio, The United States	Thomas Alva Ediso	"I have n
10	Eleanor Roosevelt	October 11, 1884	in The United States	Anna Eleanor Roos	"A woma
11	Steve Martin	14-Aug-45	in Waco, Texas, The United States	Stephen Glenn "Ste	"A day w
12					
13					
14					
15					
16					
17					
18					
19					
20					
21					

A	B	C
name	born_date	born_location
Albert Einstein	March 14, 1879	in Ulm, Germany
J.K. Rowling	31-Jul-65	in Yate, South Gloucestershire, England, The United Kingdom
Albert Einstein	March 14, 1879	in Ulm, Germany
Jane Austen	December 16, 1775	in Steventon Rectory, Hampshire, The United Kingdom
Marilyn Monroe	1-Jun-26	in The United States
Albert Einstein	March 14, 1879	in Ulm, Germany
André Gide	November 22, 1869	in Paris, France
Thomas A. Edison	February 11, 1847	in Milan, Ohio, The United States
Eleanor Roosevelt	October 11, 1884	in The United States
Steve Martin	14-Aug-45	in Waco, Texas, The United States

問題　BeautifulSoup 能「找到元素」，但元素裡的文字還要「再拆一次」，才是能計算、能分析的資料。

課堂練習 - 2



http://quotes.toscrape.com/

- 利用 BeautifulSoup 模組，爬取 Quotes to Scrape 首頁各名言與作者資訊，且將資訊存成 authors.csv 。
  欄位名稱 : name、born_date、born_location、quote、tags 。

- 在 Quotes to Scrape 首頁的 10 位作者
  - 將名言頭尾的引號去掉
  - 統計各國有幾位作者？
  - 誰最早出生?

本週一件事

節 1 ~ 3

正規表達式 × Python re 模組

- 字元類 / 量詞 / 邊界 + Python 寫法
- re.findall / search / fullmatch
- 分組、具名分組

節 1

正規表達式的基本樣式

Regular Expression Basics

字串方法，為什麼不夠用？

字串方法：靠「位置」

```
s1 = "In stock (22 available)"
s2 = "22 available"        # 格式稍微變了

s1.split("(")[1].split(" ")[0]   # '22'
s2.split("(")[1].split(" ")[0]   # IndexError
```



regex：靠「樣式」

```
import re

re.search(r"\d+", s1).group()   # '22'
re.search(r"\d+", s2).group()   # '22'
# \d+ =「一個以上的數字」
```

參考文件：

- re — Regular expression operations

概念　字串方法依賴「固定位置」；regex 描述文字的「長相」。

正規表達式 Regular Expression

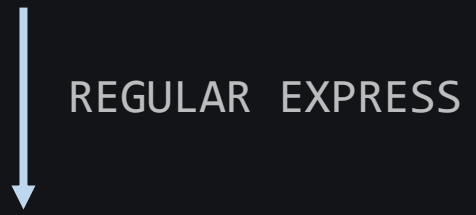
- 一種用於字串匹配和文字處理的特殊語法工具。
- 常用來搜尋、比對、取代或切割字串。

"$ 100"

" NT$ 100"

"HK$ 100 "

"¥ _ 100"



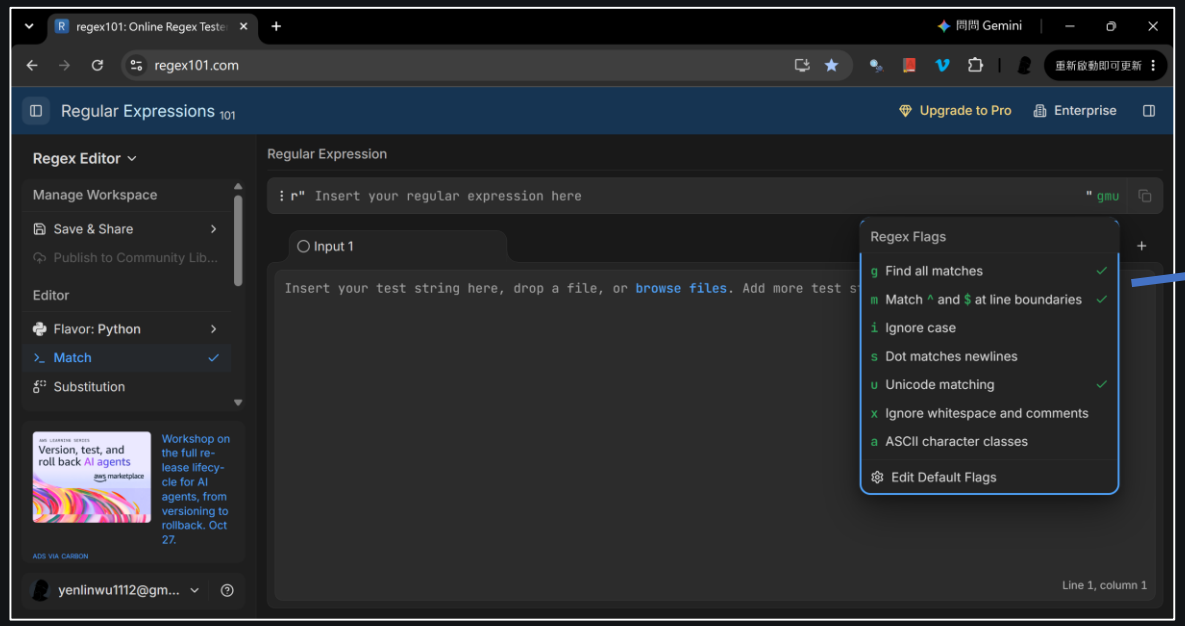
"100"

參考文件：

- Regular Expression HOWTO

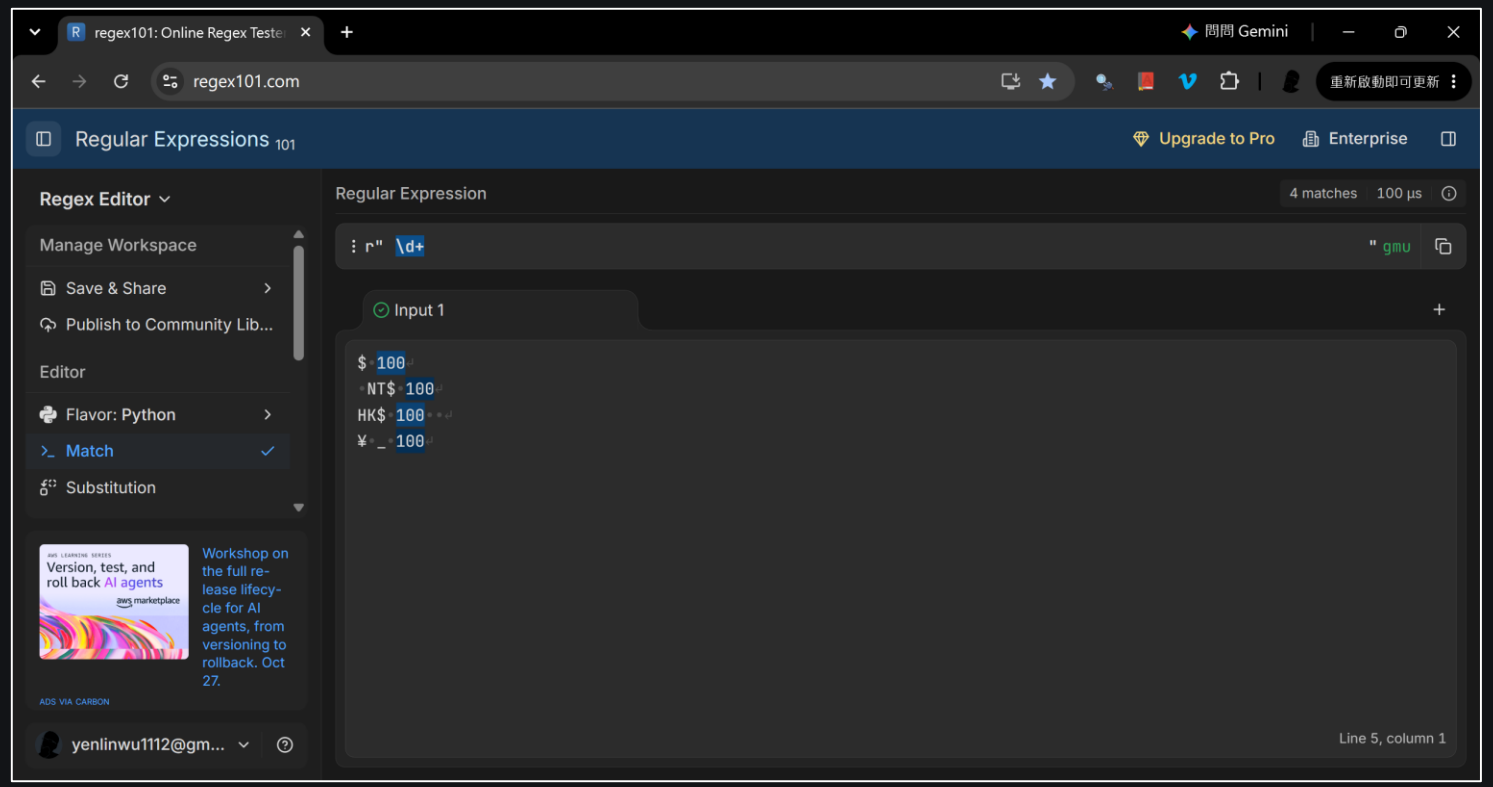
regex101 網站先驗證寫法

1. **開 regex101**
   regex101.com 線上測試
2. **選 FLAVOR**
   改成 Python
   （語法各家略異）
3. **貼測試文字**
   TEST STRING 貼實際文字
4. **輸入樣式**
   符合處即時反白
5. **看右側說明**
   EXPLANATION 逐段解釋



Flags	對應的 Python 行為
g	等於 re.findall；關掉就只標第一個，等於 re.search。
m	測試字串有多行，希望 ^ $ 每一行都生效。 Python 預設是關閉，要另外加 flags=re.M 。
u	Python 3 預設就是 Unicode（\w 會配到中文）

regex101 網站先驗證寫法



"$ 100"

" NT$ 100"

"HK$ 100 "

"¥ _ 100"

re.findall(r"\d+", string)

"100"

Python 的 re 模組

寫法	說明
import re	Python 內建，不用 pip install
r"樣式"	raw string：反斜線原封不動交給 re
re.findall(樣式, 字串)	回傳 list，列出全部配對

```python
import re

text = "Explain Like I'm 10"

result = re.findall(r"\d", text)      # findall( ) 找全部
print(result)                         # ['1', '2']   每個數字各一筆

result = re.findall(r"\d+", text)
print(result)                         # ['10']
```

重複匹配控制 Repeating Things

能精確控制某個字元或模式應該出現的次數

* 代表 0 次或多次 (預設為貪婪模式)
+ 代表至少 1 次以上
? 代表 0 次或 1 次 (作為可選項)

字元類 Character Class

寫法	說明
[A] [a-z] [A-Z] [0-9]	匹配 [ ] 裡的字元 當字元很多且連續時，使用連字號 - 簡化寫法
[^abc]	在方括號內的開頭加上脫字號 ^，代表排除這些字元，匹配「除了這些字元以外」的任意字元。
\w	匹配任意的字母、數字或底線
\W	匹配任意的非字母、非數字或非底線，等同於 [^A-Za-z0-9_]
\d	匹配任意數字，等同於 [0-9]
\D	匹配任意非數字，等同於 [^0-9]
\s	匹配任意的一個空白字元，包含 : 換行、Tab
\S	匹配任意的非空白字元
.	匹配除了換行之外的任意字元

範例

tom.wu·@·Tainan·@·1112 · 空白

[tom]	tom.wu·@·Tainan·@·1112
[tan]	tom.wu·@·Tainan·@·1112
[0-9]	tom.wu·@·Tainan·@·1112
[^@]	tom.wu·@·Tainan·@·1112
\w	tom.wu·@·Tainan·@·1112
\W	tom.wu·@·Tainan·@·1112
\d	tom.wu·@·Tainan·@·1112
\D	tom.wu·@·Tainan·@·1112
\s	tom.wu·@·Tainan·@·1112
\S	tom.wu·@·Tainan·@·1112
.	tom.wu·@·Tainan·@·1112

字元類的 Python 寫法

```
import re

string = "tom.wu @ Tainan @ 1112"

re.findall(r" [tom]", string)   # ['t', 'o', 'm']
re.findall(r"[0-9]", string)   # ['1', '1', '1', '2']
re.findall(r"\s", string)     # [' ', ' ', ' ', ' ']
re.findall(r"\W", string)     # ['.', ' ', '@', ' ', ' ', '@', ' ']
re.findall(r"\w", string)     # ['t', 'o', 'm', 'w', 'u', 'T', 'a', 'i', 'n', 'a', 'n', '1', '1', '1', '2']
```

概念　字元類一次只配「一個字」；想一次抓 tom、Tainan 這樣的一整串，要在後面加量詞。

課堂練習 - 3

A	B	C
name	born_date	born_location
Albert Einstein	March 14, 1879	in Ulm, Germany
J.K. Rowling	31-Jul-65	in Yate, South Gloucestershire, England, The United Kingdom
Albert Einstein	March 14, 1879	in Ulm, Germany
Jane Austen	December 16, 1775	in Steventon Rectory, Hampshire, The United Kingdom
Marilyn Monroe	1-Jun-26	in The United States
Albert Einstein	March 14, 1879	in Ulm, Germany
André Gide	November 22, 1869	in Paris, France
Thomas A. Edison	February 11, 1847	in Milan, Ohio, The United States
Eleanor Roosevelt	October 11, 1884	in The United States
Steve Martin	14-Aug-45	in Waco, Texas, The United States

- 在課堂練習 – 1 所得的 authors.csv 檔中，利用字元類的方法，完成下列任務：
  - born_date 的所有數字
  - born_location 的所有大寫字母
  - tags 裡所有非小寫字母的字元
  - quote 裡所有的標點符號

預期輸出

```
① ['1', '4', '1', '8', '7', '9']
② ['U', 'G']
③ [';', '-', ';', ';']
④ [' “', ',', ',', ',', '.', '” ']
```

課堂練習 - 3　參考寫法

	參考寫法	說明
①	re.findall(r"[0-9]", born)	[0-9] 等同於 \d 。
②	re.findall(r"[A-Z]", loc)	
③	re.findall(r"[^a-z]", tags)	[^a-z] 等同於 \W 。
④	re.findall(r"[^\w\s]", quote)	不是字、也不是空白。

量詞 Quantifiers

- 用來指定「前一個字元或字元類必須連續出現幾次」的符號。
- 量詞本身不能單獨存在，它必須緊跟在某個字元、字元類或群組的後面。

符號	說明
*	出現 0 次或多次
+	出現至少 1 次
?	出現 0 或 1 次
{ n }	剛好出現 n 次
{ n, }	至少出現 n 次
{ n, m }	出現 n 到 m 次之間

```
import re

string = "tom.wu @ Tainan @ 1112"

Re.findall(r" [tom]+", string)   # ['tom']
re.findall(r"[0-9]+", string)    # ['1112']
re.findall(r"\w+", string)       # ['tom', 'wu', 'Tainan', '1112']
```

量詞的 Python 寫法

```
import re

dates = """2026-10-19
           2026-1-9
           20261019
           115-10-19
           2026 - 10 - 19"""

re.findall(r"\d{4}-\d{2}-\d{2}", dates)       # ['2026-10-19']
re.findall(r"\d{4}-\d{1,2}-\d{1,2}", dates)   # ['2026-10-19', '2026-1-9']
re.findall(r"\d{4}-?\d{2}-?\d{2}", dates)     #  ['2026-10-19', '20261019']
re.findall(r"\d+-\d+-\d+", dates)             # ['2026-10-19', '2026-1-9', '115-10-19']
```

概念　同一字串資料，量詞不同結果不同：{n} 最嚴格、+ 最寬鬆。

課堂練習 - 4

A	B	C
name	born_date	born_location
Albert Einstein	March 14, 1879	in Ulm, Germany
J.K. Rowling	31-Jul-65	in Yate, South Gloucestershire, England, The United Kingdom
Albert Einstein	March 14, 1879	in Ulm, Germany
Jane Austen	December 16, 1775	in Steventon Rectory, Hampshire, The United Kingdom
Marilyn Monroe	1-Jun-26	in The United States
Albert Einstein	March 14, 1879	in Ulm, Germany
André Gide	November 22, 1869	in Paris, France
Thomas A. Edison	February 11, 1847	in Milan, Ohio, The United States
Eleanor Roosevelt	October 11, 1884	in The United States
Steve Martin	14-Aug-45	in Waco, Texas, The United States

- 在課堂練習 – 1 所得的 authors.csv 檔中，利用字元類 + 量詞，完成下列任務：
  - born_date 的完整年份
  - born_date 的月份（英文單字）
  - born_location 的每一個英文單字
  - tags 拆成一個一個標籤

預期輸出

```
① ['1879']
② ['March']
③ ['in', 'Ulm', 'Germany']
④ ['change', 'deep-thoughts', 'thinking', 'world']
```

課堂練習 - 4　參考寫法

	參考寫法	說明
①	re.findall(r"\d{4}", born)	完整年份為 4 個數字組合。
②	re.findall(r"[A-Z][a-z]+", born)	一個大寫開頭 + 一個以上的小寫。
③	re.findall(r"[A-Za-z]+", loc)	連續英文字母 = 一個單字；也可寫 \w+ 。
④	re.findall(r"[a-z-]+", tags)	字元類方括號中放 -，deep-thoughts 才不會被拆開。

中介字元 Metacharacters

符號	意思
^	字串的開頭
$	字串的結尾
\b	字的邊界（字與非字之間）
.	任意一個字（換行除外）
\	**跳脫 escape**： 如欲匹配符號本身，例如句點 .，就要在前面加 \， 告訴 re「這個符號不要當指令，照字面配」。

```
import re

string = "tom.wu @ Tainan @ 1112"

re.findall(r"\w+", string)      # ['tom', 'wu', 'Tainan', '1112']
re.findall(r"^\w+", string)    # ['tom']
re.findall(r"\w+$", string)    # ['1112']

re.findall(r"\d{2}+", string)      # ['11', '12']
re.findall(r"\b\d{4}\b", string)   # ['1112']

re.findall(r  “\w+.\w+", s)          # ['tom.wu', 'Tainan', '1112']
re.findall(r"\w+\.\w+", string)   # ['tom.wu']
```

概念　^ 只看字串的起頭、$ 只看字串的結尾、匹配符號本身要加 \ 。

課堂練習 - 5

A	B	C
name	born_date	born_location
Albert Einstein	March 14, 1879	in Ulm, Germany
J.K. Rowling	31-Jul-65	in Yate, South Gloucestershire, England, The United Kingdom
Albert Einstein	March 14, 1879	in Ulm, Germany
Jane Austen	December 16, 1775	in Steventon Rectory, Hampshire, The United Kingdom
Marilyn Monroe	1-Jun-26	in The United States
Albert Einstein	March 14, 1879	in Ulm, Germany
André Gide	November 22, 1869	in Paris, France
Thomas A. Edison	February 11, 1847	in Milan, Ohio, The United States
Eleanor Roosevelt	October 11, 1884	in The United States
Steve Martin	14-Aug-45	in Waco, Texas, The United States

- 在課堂練習 – 1 所得的 authors.csv 檔中，利用中介字元，完成下列任務：
  - born_date 的「日」（不能把年份切碎）
  - born_date 開頭的月份
  - born_location 的國家（最後一段）
  - name 裡的英文縮寫（J.K.）

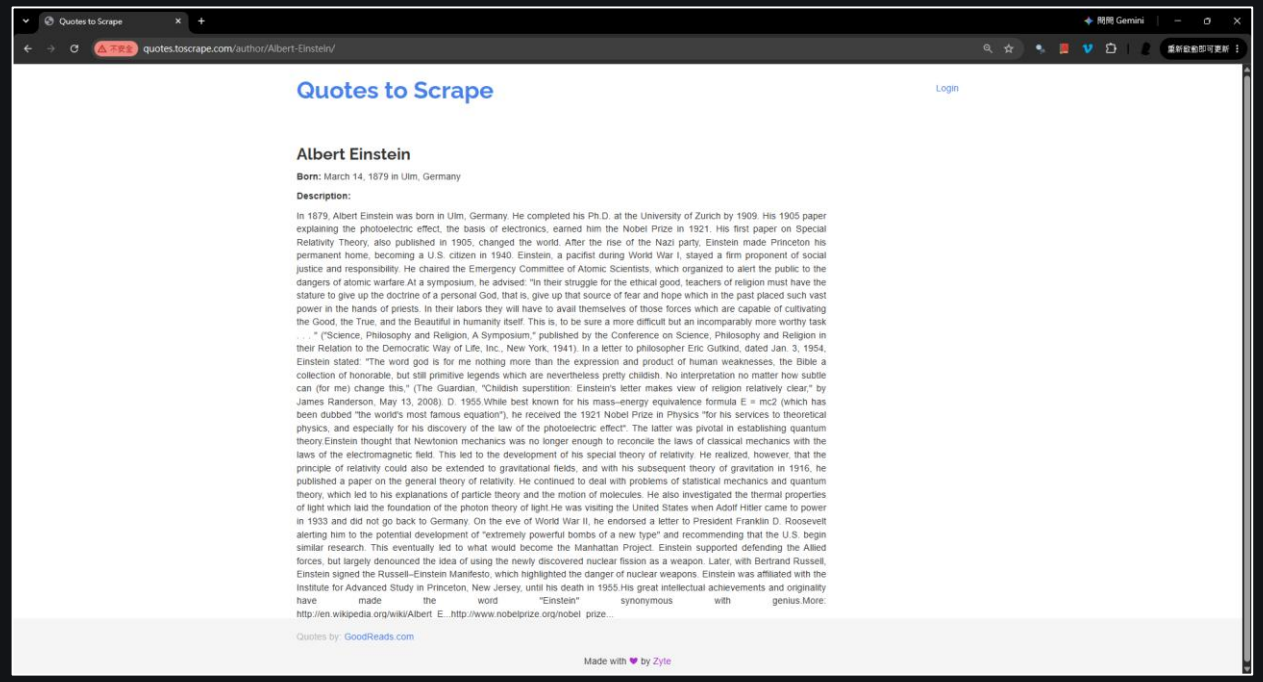
預期輸出

① ['14']
② ['March']
③ ['Germany'] / ['The United States']
④ ['J.', 'K.']

課堂練習 - 5　參考寫法

	參考寫法	說明
①	re.findall(r"\b\d{1,2}\b", born)	沒加 \b 會得到 ['14', '18', '79']，年份被切碎
②	re.findall(r"^[A-Z][a-z]+", born)	^ 限定只看字串開頭
③	re.findall(r"[A-Z][^,]*$", loc)	$ 限定到結尾都沒有逗號；Monroe 沒逗號也抓得對
④	re.findall(r"[A-Z]\.", name)	沒跳脫寫成 [A-Z]. 會多抓到 'Ro'

課堂練習 - 6



http://quotes.toscrape.com/author/Albert-Einstein/

- 爬取 Quotes of Scrape 網站的 Albert Einstein 作者頁，且萃取出下列資訊：
  ① 出生年？ (文字轉換成整數)
  ② 介紹提到的年份 (去除重複，且由小到大排序)
  ③ 去世年？ (文字轉換成整數)
  ④ 享年幾歲？

預期輸出

```
born_text = 'March 14, 1879'
desc = 'In 1879, Albert Einstein was ...'

① 1879
② ['1879', '1905', '1909', '1916', '1921', '1933',
   '1940', '1941', '1954', '1955', '2008']
③ 1955
④ 76
```

re 模組常用的函式

函式	適用情境	回傳
re.findall	匹配「全部」 例如：找出所有年份	list
re.search	匹配整個字串的「第一個」	Match / None
re.match	只匹配字串的「開頭」	Match / None
re.fullmatch	驗證「整段字串」 例如：日期格式是否正確？	Match / None

概念　匹配成功會回傳 re.Match 物件，可用 .group() 取出內容。

```
import re

string = "tom.wu @ Tainan @ 1112"

re.findall(r"\w+", string)     # ['tom', 'wu', 'Tainan', '1112']

re.search(r"\d+", string).group()    # '1112'  第一個數字串
re.match(r"\w+", string).group()    # 'tom'   開頭為字串
re.match(r"\d+", string).group()    # None  開頭非數字


# 月份英文 + 空格 + 日 + 逗號空格 + 4 位數年份
DATE = r"[A-Z][a-z]+ \d{1,2}, \d{4}"

re.fullmatch(DATE, "March 14, 1879")   # Match   格式正確
re.fullmatch(DATE, "June 01, 1926")    # Match   格式正確
re.fullmatch(DATE, "1879-03-14")       # None    格式不同
re.fullmatch(DATE, "March 14 1879")    # None    少了逗號
```

匹配不到時，回傳的是 None

常見錯誤

```
m = re.search(r"\d+", "Out of stock")
print(m)       # None

m.group()
# AttributeError: 'NoneType' object has no
# attribute 'group'
```

正確寫法：先判斷再取值

```
text = "(22 available)"

m = re.search(r"\((\d+) available\)", text)
if m:
    stock = m.group(1)

else:
    stock = 0

print(int(stock))  # 22
```

re.sub 匹配替換

```
re.sub(pattern, repl, string, count=0)
```

- pattern：正規表達式
- repl：替換的字串
- string：原始的字串內如容
- count：最大替換次數（0 表示全部替換）

```
import re

string = "<h1>文章標題</h1><p>這是第一段內容。</p>"

re.sub(r"<[^>]+>", "", string)
```

尋找以 < 開頭、> 結尾，且中間包含一個以上非 > 的任意字元



文章標題這是第一段內容。

概念　string.replace( ) 只能指定固定字串內容，re.sub( ) 則具備「模糊搜尋」與「動態判斷」能力。

課堂練習 - 7

電話號碼資訊：

```
phones = ["+886912-345-678","+886 922 111 886",
          "0928 888-666", "0966-123-345"]
```

- 利用 re.sub 整理左邊的電話號碼：
  ① 開頭的 +886 轉換成 0 。
  ② 刪掉數字以外的符號（ - 空白 . ）。
  ③ 利用 re.fullmatch 檢查清理前、後，是否為 09 開頭的 10 碼數字。
  ④ 利用 re.sub 的 count，只顯示末 4 碼。

預期輸出

```
① ['0912-345-678', '0922 111 886',
   '0928 888-666', '0966-123-345']
② ['0912345678', '0922111886',
   '0928888666', '0966123345']
③ 清理前 [False, False, False, False]
   清理後 [True, True, True, True]
④ ['******5678', '******1886',
   '******8666', '******3345']
```

分組

- 用小括號 ( ) 把一部分的規則包起來；
  整段都要匹配，但只取小括號裡的內容。
- 適用情境：從一長串符合規則的文字中，只取出其中某幾段精準的內容。

```
string = "Born: March 14, 1879  “

m = re.search(r"([A-Z][a-z]+) (\d{1,2}), (\d{4})", string)

print(m.group())       # 'March 14, 1879'              整段
print(m.groups())      # ('March', '14', '1879')       每個分組
print(m.group(3))      # '1879'                        取第 3 組


# 正規表示式有分組時，findall 回傳的是「分組內容」（tuple 清單），非整段文字。
print(re.findall(r"([A-Z][a-z]+) (\d{1,2}), (\d{4})", "March 14, 1879 / June 01, 1926"))
# [('March', '14', '1879'), ('June', '01', '1926')]
```

具名分組 Named Groups

- 語法 : (?P<name>reg_pattern)
- 在括號內加上 ?P<名字> 為分組命名後，呼叫 .groupdict( ) 會一口氣將所有命名分組，且打包成一個字典 (Dictionary) 型態回傳，其中，分組名稱是 Key，匹配文字是 Value。
- 適用情境 : 將複雜、有規律的文字，例如：Log 紀錄、API 網址、個人資料等，直接轉換為結構化的字典。

```
string = "tom.wu @ Tainan @ 1112"

m = re.search(
    r"(?P<first>\w+)\.(?P<last>\w+) @ (?P<city>\w+) @ (?P<number>\d+)",
    string
)

d = m.groupdict()
print(d)
# {'first': 'tom', 'last': 'wu', 'city': 'Tainan', 'number': '1112'}
```

課堂練習 - 8

6 行 Log 資訊

```
INFO 2026-10-20: Start crawling quotes.toscrape.com
INFO 2026-10-20: Saved 10 quotes to authors.csv
WARNING 2026-10-20: Tag not found in quote 3
ERROR 2026-10-20: Connection timed out
ERROR: 2026-10-21: Retry failed
ERROR 2026-10-21: 404 Not Found /author/Albert-Einsten
```

註 : 讀取 .log 檔的讀法與 .txt 相同

```
with open("w05.log", encoding="utf-8") as f:
    lines = f.read().splitlines()        # 一行一筆，已去掉結尾的 \n
```

- 請分析左邊 log 資訊 :
  ① 每一行拆成 dict (level、date、message)
  ② 把 dict 轉成 DataFrame
  ③ 統計各等級筆數、列出所有 ERROR 訊息

② 預期輸出

	level	date	message
0	INFO	2026-10-20	Start crawling quotes.toscrape.com
1	INFO	2026-10-20	Saved 10 quotes to authors.csv
2	WARNING	2026-10-20	Tag not found in quote 3
3	ERROR	2026-10-20	Connection timed out
4	ERROR	2026-10-21	404 Not Found /author/Albert-Einsten

今天，我們學到了什麼？

下週見 · Python 爬蟲實務應用